

PROJECT REPORT



Submitted by

ARCHANA C B : MGC21CS002

ASLAMA HISA : MGC21CS005

ROSHNA KABEER: MGC21CS011

SRIDHANYA S: MGC21CS014

Guided by

Mrs. RAJI S R

Assistant Professor Department of CSE

MG COLLEGE OF ENGINEERING

THIRUVALLAM, THIRUVANANTHAPURAM

Affiliated to APJ Abdul Kalam Technological University

Department of Computer Science and Engineering

HOSPITAL MANAGEMENT SYSTEM

MG COLLEGE OF ENGINEERING

THIRUVALLAM, THIRUVANANTHAPURAM

Approved by AICTE & Govt. of Kerala | Affiliated to APJ Abdul Kalam Technological University



CERTIFICATE

Certified that the report titled **Hospital Management System** is a Bonafide record of the mini project done by **ARCHANA CB (MGC21CS002)**, **ASLAMAHISA (MGC21CS005)**, **ROSHNA KABEER (MGC21CS011)**, **SRIDHANYA S (MGC21CS014)** under our supervision and guidance, at the department of COMPUTER SCIENCE ENGINEERING, MG COLLEGE OF ENGINEERING for the partial fulfilment for the third year of the B.Tech degree of the APJ Abdul Kalam Technological University.

MR.Vimal Raj R

Head of the Department
Dept.of Computer Science Engg.

Mrs.Raji S R

Internal Guide and Coordinator
Dept.of Computer Science Engg.

ACKNOWLEDGEMENT

We are using this opportunity to express our sincere gratitude to everyone who supported us throughout this mini project. We first like to thank the almighty for showering his blessings upon us.

We express our sincere gratitude towards our Head of the Department of Computer Science Engineering, Mr. VIMAL RAJ for providing an opportunity for undertaking this project.

We would also like to thank our project internal guide, Mrs. RAJI SR Assistant professor, for all the support given by her throughout the completion of the project.

Last but not the least, we would like to express our profound thanks to all our friends for their support. We thank all the above mentioned people for their aspiring guidance, invaluable constructive criticism and friendly advice during the project work. We are sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to the project.

WITH GRATITUDE,
ARCHANA C B
ASLAMA HISA
ROSHNA KABEER
SRIDHANYA S

ABSTRACT

Our project Clinic Management System is a highly useful software for doctors as well as patients. The system aims to enhance the efficiency and effectiveness of clinic workflows, help communication between doctors and patients, and provide a seamless healthcare experience online. It consists of three modules: Admin, Doctor and Patient. It is a standalone system which can be used with valid login credentials. The admin supervises the working of doctor module, the admin can add the doctor as well as view the doctors list. The doctor can use valid login credentials which is obtained after a successful registration to login into his/her profile. Same is the process for patient as well. The doctor can monitor the medical history of patients and stay informed about their upcoming appointments. The patient can book an appointment, get information about the doctor and different departments available. By leveraging technology and streamlining administrative tasks, this system contributes to enhanced patient care, efficiency, and health outcomes.

CONTENT

1. INTRODUCTION	1
2. INTRODUCTION TO SYSTEM ANALYSIS	2
3. EXISTING SYSTEM	3
3.1 LIMITATIONS OF EXISTING SYSTEM	3
4. PROPOSED SYSTEM	4
4.1 ADVANTAGES OF PROPOSED SYSTEM	4
4.2 FEATURES OF PROPOSED SYSTEM	4
5. REQUIREMENT SPECIFICATIONS	5
5.1 HARDWARE	5
5.2 SOFTWARE	5
6. FUNCTIONAL SPECIFICATIONS	6
6.1 ADMIN MODULE	6
6.2 PATIENT MODULE	7
6.3 DOCTOR MODULE	8
7. NON-FUNCTIONAL SPECIFICATIONS	9
8. FEASIBILITY STUDY	10
8.1 ECONOMIC FEASIBILITY	10
8.2 OPERATIONAL FEASIBILITY	11
8.3 TECHNICAL FEASIBILITY	11
9. SYSTEM DESIGN	12
9.1 DESIGN OF SUBSYSTEM	12
9.1.1 ADMIN	12
9.1.2 PATIENT	12
9.1.3 DOCTOR	13
10. USER INPUT DESIGN	14
10.1 INPUT DESIGN	14
10.2 DATABASE DESIGN	15
11. ENTITY RELATIONSHIP DIAGRAM	16
12. ARCHITECTURAL DESIGN	17
13. HARDWARE SPECIFICATION	18
14. SOFTWARE SPECIFICATION	20
15. CODING	22
16. TESTING	34
16.1 UNIT TESTING	34
16.2 INTEGRATION TESTING	35
16.3 TYPES OF TEST CASE	35
16.4 SYSTEM TESTING	35
17. SECURITY, BACKUP & RECOVERY	36
17.1 SECURITY	36
17.2 BACKUP	36
17.3 RECOVERY	37
18. FUTURE ENHANCEMENT	38
19. SOFTWARE MAINTENANCE	39
20. CONCLUSION	40
21. BIBLIOGRAPHY	41

INTRODUCTION

Our project Clinic Management System is a highly useful software for doctors as well as patients. The system aims to enhance the efficiency and effectiveness of clinic workflows, help communication between doctors and patients, and provide a seamless healthcare experience online. It consists of three modules: Admin, Doctor and Patient. It is a standalone system which can be used with valid login credentials. The admin supervises the working of doctor module, the admin can add the doctor as well as view the doctors list. The doctor can use valid login credentials which is obtained after a successful registration to login into his/her profile. Same is the process for patient as well. The doctor can monitor the medical history of patients and stay informed about their upcoming appointments. The patient can book an appointment, get information about the doctor and different departments available. By leveraging technology and streamlining administrative tasks, this system contributes to enhanced patient care, efficiency, and better health outcomes.

INTRODUCTION TO SYSTEM ANALYSIS

Our Clinic Management System is a valuable tool that facilitates online doctor appointment bookings for patients. We propose an integrated doctor-patient management system designed to enhance both doctors' workflow and patients' healthcare experience. Our system empowers doctors to efficiently manage their appointment schedules online, with patients able to reserve available slots conveniently. It meticulously handles appointment data across various dates and times. Every patient visit results in the secure storage of their medical records in the database, ensuring accessibility for future reference. Patients can easily access their entire medical history when needed, while doctors have the ability to review patients' medical records even before their visits, streamlining patient-doctor interactions through an intuitive online interface. Additionally, doctors can update patient informations and medical conditions during each visit, and they have the ability to track their entire patient roster for any given calendar day. Our system operates as a standalone platform, accessible via valid login credentials, providing an efficient and user-friendly solution for healthcare.

EXISTING SYSTEM

In the present system, doctors cannot view all the details in any other device. The doctor cannot store the details regarding the medications prescribed to a particular patient. Doctors don't have access to their patient's details, and they need to go for various documents and files in order to access the data. Existing system of getting information via files takes so much time, that sometimes lead to losing up of some important moments of the patient's life. The user may not get desired information and maybe misguided as well. Existing system takes too much time and effort.

Limitations of Existing System

- It takes lot of time to access information.
- It sometimes leads to wrong information and thus mislead us.
- The details cannot be viewed on any other device.
- The facility in existing system is limited so that we will have to find information from multiple sources to get desired information and so it takes a lot of effort
- Lack of security for records and database.
- The availability of doctor is unknown to the patients.

PROPOSED SYSTEM

In this system, doctors enjoy the convenience of accessing patient medical histories from their devices whenever the need arises. It operates as a secure and flexible standalone system. Its standout feature is the swift retrieval of urgent information, making it a valuable tool for timesensitive situations. The proposed project presents a highly automated and user-friendly appointment booking system, streamlining the process of finding specialized doctors and scheduling appointments online. For patients and users, it offers a simple and efficient way to search for doctors based on their specialization and book appointments. Simultaneously, it provides a platform for doctors to create profiles, enabling patients to easily locate them and secure appointments. This web-based application resolves the complexities of managing and booking appointments according to user preferences, alleviating the manual burden of doctors in allocating appointments. It's an effective solution where users can select their preferred doctor and book appointments with ease.

Advantages of Proposed System:

- Patient does not have to wait in queue to book an appointment.
- This system saves time of both doctor and patient.
- Patient can view his medical history whenever required.
- Doctor can view a patient's medical reports before next consultation.
- Patient does not have to carry medical prescription whenever he visits the doctor.
- Doctor can view his appointments at anywhere at any time.

Features of Proposed System :

- System reduces time consumption for getting an appointment with a particular doctor, This it gives the advantage of booking appointment from home.
- The medical history of a particular patient can be obtained through this system whenever the doctor needs it.
- It is a standalone system which can be used by both doctor and patient with valid login credentials.

REQUIREMENT SPECIFICATIONS

Hardware Requirements :

- Processor : Intel core i7
- Processor speed : 3GHz or above
- RAM : 3GB or above
- Hard Disk Capacity : 1TB
- Keyboard : Multimedia Keyboard
- Mouse : Standard
- USB : 2.0 & 3.0

Software Requirements :

- Operating System : Windows 11
- Front End : HTML 5, CSS
- Backend : Node JS
- Language : JavaScript
- Tools used : Visual Studio Code
- Database : MongoDB

FUNCTIONAL SPECIFICATIONS

ADMIN MODULE

The admin module allows admin to login and view the details of doctors and patients registered in the respective modules. First, admin will be visiting the home page. There is a pop-up window leading to the login for admin, doctor and patient. The admin should click the respective button that leads to the admin login. The admin successfully login into his profile and he/she can view the details of registered doctors and patients.

FN 1: Login

Admin can login with the given user id and password.

Input : Enter Username and Password.

Output : Login Successfully and redirects to profile of admin.

FN 2: View Patients

Admin can view the details of registered patients.

Input : Enter Name, Age, Phone number, Username, Password

Output : View details of patients.

FN 3: Manage Doctors

Admin can view the details of registered doctors.

Input : Enter Name, department, username, password.

Output : View details of doctors.

FN 4: View Scheduled Appointments

Input : Enter Name, Age, Doctor, Date.

Output : View details of scheduled appointments

PATIENT MODULE

The patient module allows patients to register, login, and log out. The patient is one who gets the best experience from the system. First, patient will be visiting the home page. There are separate pop-up options in navbar leading to the login/registration portal for admin, doctor and patient. The patient should click the respective button leading to patient login. If not registered, the patient should click the sign-up provision on navbar and can register and login afterwards. The patient successfully login into his profile and he can view the details of registered doctors and can book appointment.

FN 1: Registration

A patient can register into the software by giving valid credentials.

Input : Enter Name, Age, Phone number, Username, Password.

Output : New user added.

FN 2: Login

It allows registered applicant to login with the given user id and password.

Input : Enter Username and Password.

Output : Login Successfully and redirects to profile of patient.

FN 3: View Doctor

Patient can view the details of available doctors.

Input : Enter Name, department, username, password

Output : View details of available doctors

FN 4: Booking Appointment

It is used to book an appointment with the respective doctor.

Input : Enter Name, Age, Doctor, Date.

Output : Booking Successful and provision to logout

DOCTOR MODULE

The doctor module allows doctors to register, login, view patient details and upcoming appointments, and logout. First, doctor will be visiting the home page. There are separate popup windows leading to the login/registration portal for admin, doctor and patient on the navbar. The doctor should click the respective button leading to doctor login. If not registered, the doctor should click the sign-up provision on navbar and register and login afterwards. The doctor successfully login into his profile and he/she can view the details of patients and upcoming appointments.

FN 1: Registration

A doctor can register into the software to access patient details and appointment schedule.

Input : Enter Name, department, username, password.

Output : New user added.

FN 2: Login

It allows registered applicant to login with the given user id and password.

Input : Enter Username and Password.

Output : Login Successful and redirects to profile of doctor.

FN 3: View Patients

Doctors can view the details of patients.

Input : Enter Name, Age, Phone number, Username, Password .

Output : View details of patients.

FN 4: View Upcoming Appointments

Doctors can view the details of upcoming appointments.

Input : Enter Name, Age, Doctor, Date

Output : View details of upcoming appointments.

Non-Functional Requirements

In systems, a non-functional requirement (NFR) is a requirement that specifies criteria that can be used to judge the operation of a system, rather than specific behaviour. They are contrasted with functional requirements that define specific behaviour or functions:

- Usability:

Usability defines how difficult it will be for a user to learn and operate the system. Efficiency of use: the average time it takes to accomplish a user's goals, how many Tasks a user can complete without any help, the number of transactions completed Without errors.

- Authenticity:

The system meets authenticity as only those users with a valid username and password can log into the application.

- Data integrity:

Data integrity is ensured by only the accurate data.

- Performance:

It ensures good performance by maintaining a low response time, high utilization and fast throughput.

- Availability:

The application meets availability as a user can access it any time.

- Security:

The data is securely stored in database and only the authorized user can access it.

FEASIBILITY STUDY

Feasibility study is the test of the system proposal made to identify whether the user needs may be satisfied using the current software and hardware technologies, whether the system will be cost effective from a business point of view and whether it can be developed with the given budgetary constraints. A feasibility study should be relatively cheap and done at the earliest possible time. Depending on the study, the decision is made whether to go ahead with a more detailed analysis. When a new project is proposed, it normally goes through feasibility assessment. Feasibility study is carried out to determine whether the proposed system is possible to develop with available resources and what should be the cost consideration. Facts considered in the feasibility analysis were:

- Economic Feasibility
- Operational Feasibility
- Technical Feasibility

Economic feasibility

This feasibility study presents tangible and intangible benefits from the project by comparing the development and operational cost. This system needs some more initial investment than the existing system, but it can be justifiable that it will improve quality of service.

Thus, feasibility study should centre along the following points:

- Improvement resulting over the existing method in terms of accuracy, timeliness.
- Cost comparison
- Estimate on the life expectancy of the hardware
- Overall objective

Our project is economically feasible. It does not require much cost to be involved in the overall process. It is essential because the main goal of the proposed system is to have economically better results with the increase.

Operational Feasibility

Operational analysis is the most frequently used method for evaluating the effectiveness of a new system. More commonly known as cost/benefit analysis, the procedure is to determine the benefits and saving that are expected from a candidate system and compare them with costs. If benefits outweigh costs, then the decision is made to design and implement the system. An entrepreneur must accurately weigh the cost versus benefits before taking an action. Cost-based study: It is important to identify cost and benefit factors, which can be categorized as follows:

1. Development costs.
2. Operating costs.

This is an analysis of the costs to be incurred in the system and benefits derivable out of the system. Time-based study: This is an analysis of the time required to achieve a return on investments. The future value of a project is also a factor. This application can be operated by all the users across the world.

Technical feasibility

Technical Feasibility deals with the hardware as well as software requirements. Technology is not a constraint to type system development. We have to find out whether the necessary technology, the proposed equipment has the capacity to hold the data, which is used in the project, should be checked to carry out this technical feasibility. The technical feasibility issues usually raised during the feasibility stage of investigation includes

- This software is running in Windows 11.
- The minimum hardware required is 3GB and maximum hardware required is 400GB.
- The system can be expanded

SYSTEM DESIGN

DESIGN OF SUBSYSTEM

A Design of Subsystem encapsulates behaviour, providing explicit and formal interfaces, and does not expose its internal contents. This provides the ability to completely encapsulate the interactions of a number of classes and/or subsystems. This project has three major modules. These three modules are the core functionalities of the system. The project is divided into three major modules according to their functionalities.

- ADMIN
- PATIENT
- DOCTOR

ADMIN :

The main purpose of this module is to view all the registered doctors and registered patients.

- Admin can add as well as view all the registered doctors.
- Admin can view all the registered patients.
- Admin can view all the scheduled appointments.

PATIENT :

The main purpose of this module is that the patient can view the details of available doctors, and book appointments.

- Patient can view the details of available doctors.
- Patient can book appointments.
- Patient can view patient history.

DOCTOR

The main purpose of this module is that the doctor can view the details of patients and keep records of his/her appointments.

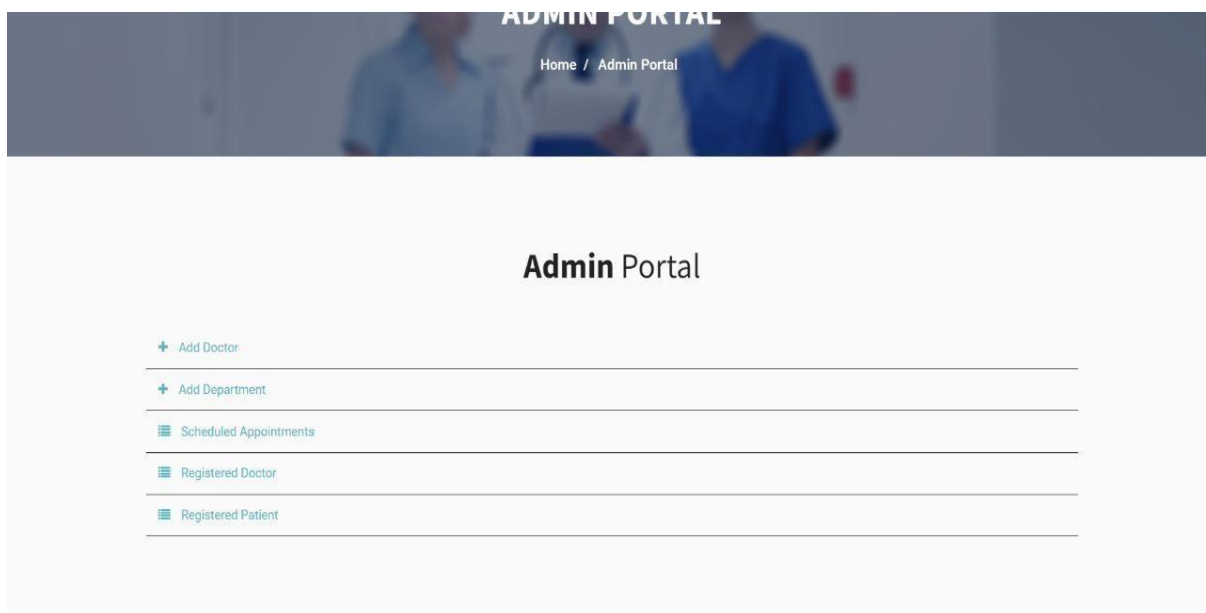
- Doctor can view the appointments of patients.
- Doctor can view and update the medical history of patients

USER INTERFACE DESIGN

Input Design :

Input designing is the basic theory to be considered during system study. The input media used in the system is the keyboard. Details are entered in the system through different data entry screens. The system is designed in a user-friendly manner. Appropriate error messages are displayed when a false data is entered. The user interface design is very important for any application. The interface design defines how the software communicates within itself, to system that interpreted with it and with human who use it. The input design is the process of converting the user-oriented description of inputs into a programmer-oriented specification. It is the link that ties the system into the world of its users. Input design involves determining the record media, method of input, speed of capture and entry to the system. The analyst should consider the following points when designing the input:

- Nature of the input processing.
- Flexibility and thoroughness validation rules.
- Handling of priorities with the input procedures.
- Use of composite input documents to reduce the number of different ones.



DATABASE DESIGN :

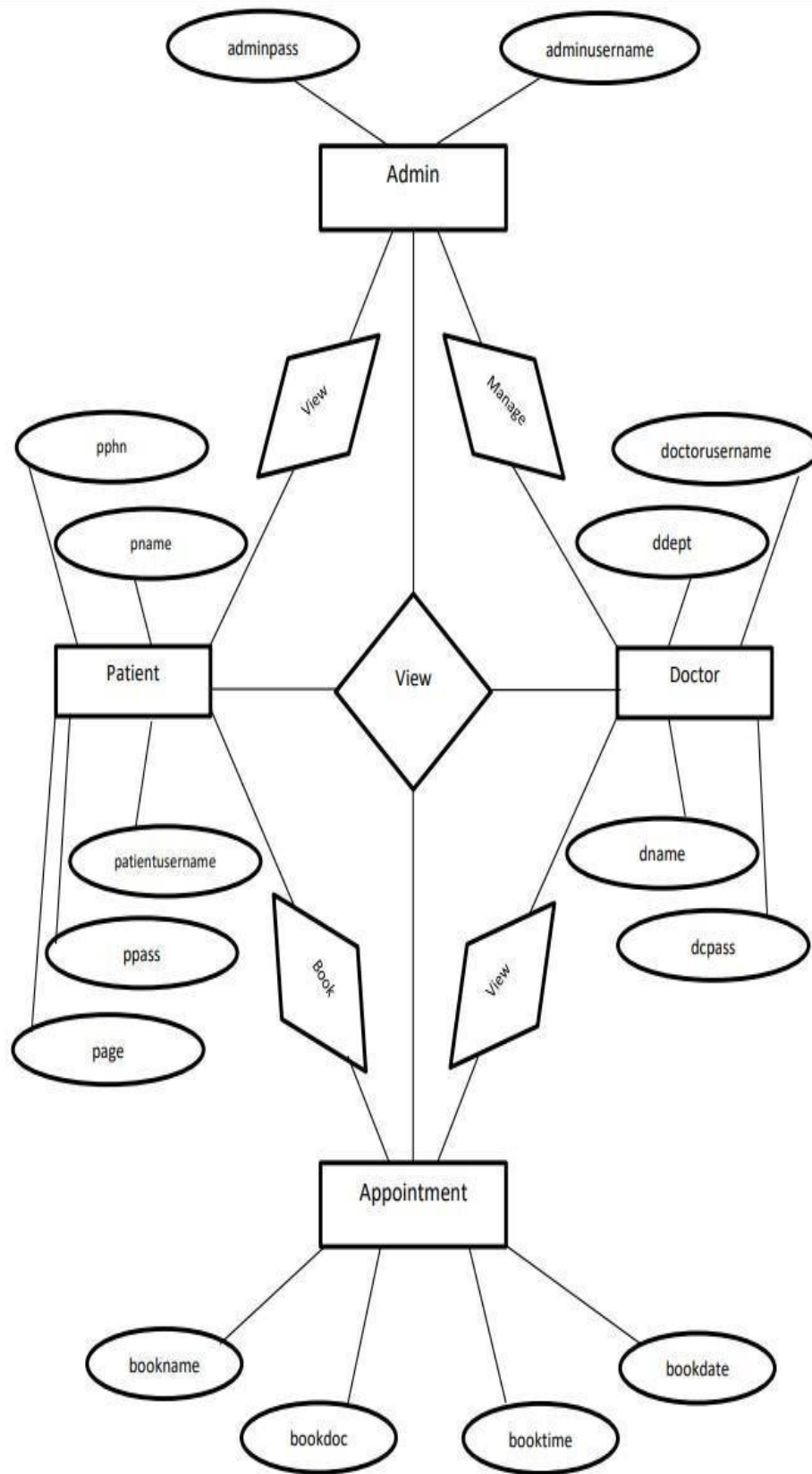
The overall objective in the development of database technology has been to treat data as an organizational resource and as an integrated whole. DBMS allow data to be protected and organized separately from other resources. Database is an integrated collection of data. The most significant form of data as seen by the programmers is data as stored on the direct access storage devices. This is the difference between logical and physical data.

Database files are the key source of information into the system. It is the process of designing database files, which are the key source of information to the system. The files should be properly designed and planned for collection, accumulation, editing and retrieving the required information. The organization of data in database aims to achieve three major objectives:

- Data integration.
- Data integrity.
- Data independence

The proposed system stores the information relevant for processing in the MongoDB database. MongoDB is an open-source NoSQL database management program. NoSQL is used as an alternative to traditional relational databases. NoSQL databases are quite useful for working with large sets of distributed data. The NoSQL DBMS uses a single master architecture for data consistency, with secondary databases that maintain copies of the primary database. Operations are automatically replicated to those secondary databases for automatic failover. MongoDB makes use of records which are made up of documents that contain a data structure composed of field and value pairs. Documents are the basic unit of data in MongoDB. The documents are similar to JavaScript Object Notation, but use a variant called Binary JSON (BSON). The benefit of using BSON is that it accommodates more data types. The fields in these documents are similar to the columns in a relational database. Values contained can be a variety of data types, including other documents, arrays and arrays of documents. Documents will also incorporate a primary key as a unique identifier. Sets of documents are called collections, which function as the equivalent of relational database tables. Collections can contain any type of data, but the restriction is the data in a collection cannot be spread across different databases.

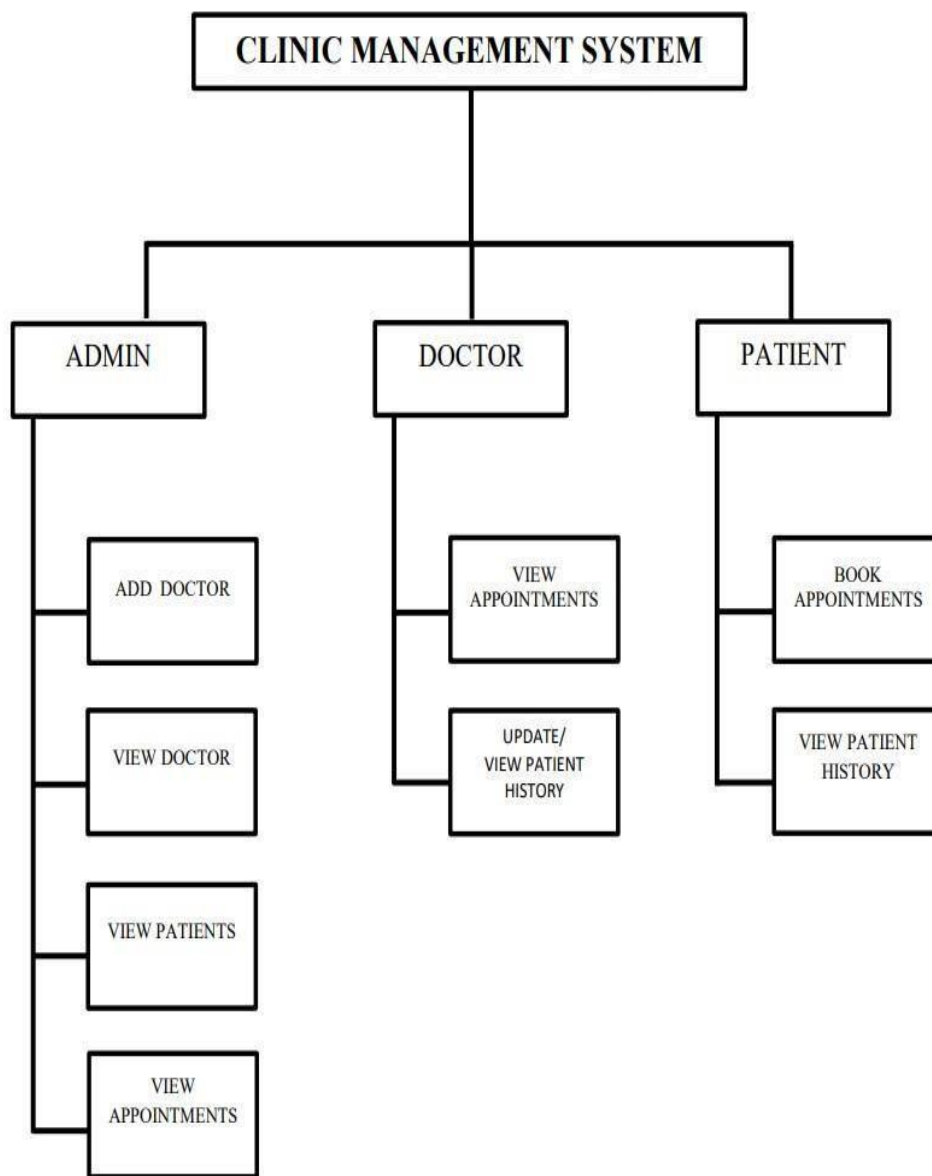
ENTITY RELATIONSHIP DIAGRAM



ARCHITECTURAL DESIGN

Structure Chart :

A structure chart (SC) in software engineering and organizational theory, is a chart which shows the breakdown of a system to its lowest manageable levels. They are used in structure programs to arrange modules into a tree. Each module is represented by a box, which contains the module's name. The tree structure visualizes the relationships between the modules.



HARDWARE SPECIFICATIONS

Intel core i7 Processor

An Intel Corei7 is the fastest version of the Intel processor for consumer-end computers and devices. Like the Intel Corei5, the Corei7 is embedded with Intel Turbo Boost Technology. The Intel Corei7 is available in two- to six-core varieties, and can support up to 12 different threads simultaneously. Its processor clock speed ranges from 1.70 GHz to up to 3.90 GHz, with cache memory from 4 to 12 MB. Intel Corei7 thermal design power (TDP) range goes from 130 watts TDP to as low as 15 watts TDP. A Core i7 will typically be better for multitasking, media-editing and media-creation tasks, high-end gaming, and similar demanding workloads.

Processor speed

With technology, increased productivity goals, faster internet, and more devices, we've created a need for speed wherever we go. We're used to getting results instantaneously and expect our devices to keep up with our requests as we multi-task our way through life. Computer processors and their clock speed are two features we most commonly associate with highperforming, fast technology. Computer processor speed (CPU speed) is one of the most important elements to consider when comparing computers. The CPU is often referred to as "the brain" of your computer, so ensuring it's working properly is very important to the longevity and functionality of your computer. Understanding what makes a good processor speed starts with understanding what exactly a processor does - and what its components do to improve the functionality of your computer.

RAM

RAM (Random Access Memory) is the hardware in a computing device where the operating system (OS), application programs and data in current use are kept so they can be quickly reached by the device's processor. RAM is the main memory in a computer. This memory stores a temporary data. Temporary data means as long as your computer system is running, your data is stored in RAM and as soon as your computer is shut down, the data stored in RAM gets deleted. RAM is volatile in nature, which means, the data is lost when the device is switched off. RAM is known as the Primary memory of the computer. RAM is known to be expensive since the memory can be accessed directly. RAM is the fastest memory; therefore, it is an internal memory for the computer.

Hard Disk Capacity

A computer hard disk drive (HDD) is a non-volatile data storage device. Non-volatile refers to storage devices that maintain stored data when turned off. All computers need a storage device, and Hard Disk Drive are just one example of a type of storage device. HDD are usually installed inside desktop computers, mobile devices, consumer electronics and enterprise storage arrays in data centres. They can store operating systems, software programs and other files using magnetic disks.

Keyboard

A multimedia keyboard is one with media keys — additional buttons, typically along the top, for controlling audio playback, for starting common applications (e.g., e-mail client and Web browser) and other auxiliary functionality. Many such keyboards also contain a volume knob, implemented as a rotary encoder so as to be able to provide relative volume changes regardless of the volume level set by the user in the operating system and application software.

Mouse

A mouse is a small device that a computer user pushes across a desk surface in order to point to a place on a display screen and to select one or more actions to take from that position. The mouse first became a widely-used computer tool when Apple Computer made it a standard part of the Apple Macintosh.

USB

USB 2.0, also known as hi-speed USB was introduced in 2000. It is an updated version of USB 1.1, which provides improved functionalities and better speed. It is capable to deliver the maximum transfer speed of 480 Megabits per second. However, practically it is approximately 280 Mbps. USB 3.0, also known as SuperSpeed USB was first made available in November 2009. It is a much-improved version of USB 2.0. It supports the data transfer rate of 5 Gigabits per second, which is much faster than the speed provided by USB 2.0 .

SOFTWARE SPECIFICATIONS

Windows 11

Windows 11 is the latest major release of Microsoft's Windows NT operating system, released in October 2021. It is a free upgrade to its predecessor, Windows 10 (2015), available for any Windows 10 devices that meet the new Windows 11 system requirements. Windows 11. A version of the Windows NT operating system.

HTML 5

HTML5 is a markup language used for structuring and presenting content on the World Wide Web. It is the fifth and final major HTML version that is a World Wide Web Consortium recommendation. The current specification is known as the HTML Living Standard.

CSS

Cascading Style Sheets is a style sheet language used for describing the presentation of a document written in a markup language such as HTML or XML. CSS is a cornerstone technology of the World Wide Web, alongside HTML and JavaScript.

Node JS

Node.js is an open-source, cross-platform, back-end JavaScript runtime environment that runs on a JavaScript Engine and executes JavaScript code outside a web browser, which was designed to build scalable network applications. Node.js is primarily used for nonblocking, event-driven servers, due to its single-threaded nature. It's used for traditional web sites and back-end API services, but was designed with real-time, push based architectures in mind.

JavaScript

JavaScript, often abbreviated JS, is a programming language that is one of the core technologies of the World Wide Web, alongside HTML and CSS. As of 2022, 98% of websites use JavaScript on the client side for web page behaviour, often incorporating third-party libraries. JavaScript runs on the client side of the web, which can be used to design / program how the web pages behave on the occurrence of an event. JavaScript is an easy to learn and also powerful scripting language, widely used for controlling web page behaviour. It is used to enhance HTML pages and is commonly found embedded in HTML code. JavaScript is an interpreted language. Thus, it doesn't need to be compiled. JavaScript renders web pages in an interactive and dynamic fashion.

Visual Studio Code

Visual Studio Code, also commonly referred to as VS Code, is a source-code editor made by Microsoft for Windows, Linux and macOS. Features include support for debugging, syntax highlighting, intelligent code completion, snippets, code refactoring, and embedded Git. Visual Studio Code is a lightweight but powerful source code editor which runs on your desktop and is available for Windows, Mac-OS and Linux. It comes with built-in support for JavaScript, TypeScript and Node.

MongoDB

MongoDB is an open source NoSQL database management program. NoSQL is used as an alternative to traditional relational databases. NoSQL databases are quite useful for working with large sets of distributed data. MongoDB is a tool that can manage document-oriented information, store or retrieve information. MongoDB supports various forms of data. It is one of the many nonrelational database technologies that arose in the mid-2000s under the NoSQL banner -- normally, for use in big data applications and other processing jobs involving data that doesn't fit well in a rigid relational model. Instead of using tables and rows as in relational databases, the MongoDB architecture is made up of collections and documents

CODING

Admin Login

```
function adminlog() {
    event.preventDefault() let data = {} data.email =
document.getElementById("username").value;
data.pass
document.getElementById("pass").value; data =
JSON.stringify(data); const xhr = new
XMLHttpRequest();
xhr.withCredentials=true; xhr.addEventListener("readystatechange", function ()
{ if
(this.readyState === this.DONE) { const res
= JSON.parse(this.responseText);
console.log(res); if(res.status == 200){ alert("Succesfully
logged in"); localStorage.user = JSON.stringify(res.user);
window.location = "admin.html"
}else{ alert('invalid login
details');
}
}
}
}
)
;
```



```
xhr.open("POST","http://localhost:3000/adminlogin");
xhr.setRequestHeader("Accept", "*/*"); xhr.setRequestHeader("Content-
Type", "application/json"); xhr.send(data);
```

```
}
```

Doctor Registration

```
function doclog() { event.preventDefault(); let
emailInput = document.getElementById("email");
let email = emailInput.value; const emailRegex =
/^[^s@]+@[^s@]+\.[^s@]+$/; if
(!emailRegex.test(email)) { alert("Please enter a
valid email address"); emailInput.focus();
return; } let data = {}; data.name =
document.getElementById("docname").value;
data.department =
document.getElementById("dep").value;
data.email = email; // Use the validated email
data.password =
document.getElementById("password").value;
data = JSON.stringify(data); const xhr = new
XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange",
function ()
{ if
(this.readyState === this.DONE) { console.log(this.responseText);
} });
xhr.open("POST", "http://localhost:3000/signupdoctor");
xhr.setRequestHeader("Accept", "/");
xhr.setRequestHeader("Content-Type", "application/json");
xhr.send(data); alert("Doctor added successfully");
}
```

Patient Registration

```
function Register() { event.preventDefault();
const name =
document.getElementById("name").value;
const age =
document.getElementById("age").value;
const email =
document.getElementById("email").value;
const password =
document.getElementById("psw").value;
const confirmPassword =
document.getElementById("confirmPsw").value;
if
(!/^d+$/i.test(age) || parseInt(age) < 1 || parseInt(age) > 150) { alert("Please
enter a valid age between 1 and 150."); return;
} const emailPattern = /^[^s@]+@[^s@]+\.[^s@]+$/; if
(!emailPattern.test(email)) { alert("Please enter a valid email address."); return; }
if (password !== confirmPassword) { alert("Passwords do not match. Please
confirm your password."); return; } const data = { name: name, age: age, email:
email, password: password

}; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;

xhr.addEventListener("readystatechange", function ()
{ if (this.readyState === this.DONE) {
console.log(this.responseText);
} }); xhr.open("POST",
"http://localhost:3000/signuppatient");
xhr.setRequestHeader("Accept", "/");
```

```
xhr.setRequestHeader("ContentType", "application/json");
xhr.send(JSON.stringify(data)); alert("User created");
}
```

Booking Appointment

```
fdocument.getElementById("name").value    =    JSON.parse(localStorage.user).name;

document.getElementById("email").value = JSON.parse(localStorage.user).email;

// Get the department and doctor select elements  const
departmentSelect  =  document.getElementById("all-dept");
const doctorSelect = document.getElementById("doc");    //
Update the doctor select options based on the selected
department      departmentSelect.addEventListener("change",
function    ()    {      const    selectedDepartment    =
departmentSelect.value; doctorSelect.innerHTML = ""; const
_opt = document.createElement("option");  _opt.value = "";
_opt.text =
"wait a min"; doctorSelect.appendChild(_opt); const xhr =
new XMLHttpRequest();  xhr.withCredentials = true;
const dept = document.getElementById("all-dept");
xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) {
doctorSelect.innerHTML = "";
_opt.value = "";
_opt.text    =    "Select    a    doctor";
doctorSelect.appendChild(_opt); const res =
JSON.parse(this.responseText);
```

```

res.data.forEach((i) => {

doctorSelect.innerHTML += `<option
value="${i._id}">${i.name}</option>`;

});

} });

xhr.open("POST",          "http://localhost:3000/getdeptdoc");
xhr.setRequestHeader("Accept",          "/");
xhr.setRequestHeader("Content-Type",    "application/json");
const    data    =    JSON.stringify({    department:
selectedDepartment }); xhr.send(data);

});

var ModalDate = document.getElementById("datepicker"); var
CurrentDate = new Date();

ModalDate.min = CurrentDate.toISOString().split('T')[0]; const
MaxDate = new Date(CurrentDate);

MaxDate.setDate(MaxDate.getDate() + 28);

ModalDate.max = MaxDate.toISOString().split('T')[0];

</script>

<!--Apoinment backend-->

<script> const
TimeSlots = {

"slot1":"8:30AM - 12:30PM",

"slot2":"1:30PM - 4:30PM",

"slot3":"5:30PM - 8:30PM"

} function appointment()

{ // event.preventDefault(); let data = {};

```



```

data.name = document.getElementById("name").value;

data.phone = document.getElementById("phone").value;

data.date = document.getElementById("datepicker").value;

data.email = document.getElementById("email").value;

data.dept = document.getElementById("all-dept").value;

data.doc = document.getElementById("doc").value;

data.message = document.getElementById("message").value;

data.time =

TimeSlots[document.getElementById("time").value]; data =

JSON.stringify(data); const xhr = new XMLHttpRequest();

xhr.withCredentials = true;

xhr.addEventListener("readystatechange", function () { if

(this.readyState === this.DONE) {

console.log(this.responseText); alert(" Appointment Requested

successfully");

} });

xhr.open("POST", "http://localhost:3000/appointment");

xhr.setRequestHeader("Accept", "/"); xhr.setRequestHeader("Content-

Type", "application/json"); xhr.send(data);

} allDept(); function allDept() { const xhr = new XMLHttpRequest();

xhr.withCredentials = true; const dept = document.getElementById("alldept");

xhr.addEventListener("readystatechange", function () { if

(this.readyState === this.DONE) { const res =

JSON.parse(this.responseText); res.data.forEach((i) => {

dept.innerHTML += `<option value="${i.name}">${i.name}</option>`;

}); } });

```

```

xhr.open("GET", "http://localhost:3000/getdept");
xhr.setRequestHeader("Accept", "/"); xhr.setRequestHeader("Content-
Type", "application/json"); xhr.send(null);
}

```

Patient History

```

let Appoinment = null; (async () => { const urlParams = new
URLSearchParams(window.location.search); const appid =
urlParams.get('id'); if (!appid) history.back(); else { await
fetch(`http://localhost:3000/viewappoinment/?id=${appid}`).then
e => e.json()).then(data => { Appoinment = data.data; });
document.getElementById('name').value = Appoinment.name;
document.getElementById('phone').value = Appoinment.phone;
document.getElementById('Btn-
addHistory').removeAttribute('disabled');
} await fetch(`http://localhost:3000/gethistory/?id=${appid}`).then(e
=> e.json()).then(data => { data = data.data; const
history = document.getElementById('history'); if
(!data.length) { let _tr = document.createElement('tr');
let _td = document.createElement('td');
_td.innerHTML = 'No data found'; _td.colSpan = 5;
_tr.appendChild(_td); history.appendChild(_tr); }
else { data.forEach((i, j) => { let _tr =
document.createElement('tr'); let _td =
document.createElement('td'); _td.innerHTML = j +
1; _tr.appendChild(_td); _td =
document.createElement('td'); _td.innerHTML =

```

```

i.name; _tr.appendChild(_td); _td =
document.createElement('td'); _td.innerHTML =
i.phone; _tr.appendChild(_td);

_td = document.createElement('td'); const
Cdate = new Date(i.createdAt);

_td.innerHTML = `${Cdate.getDate()}-${Cdate.getMonth()}- ${Cdate.getFullYear()}
${Cdate.getHours()}:${Cdate.getMinutes()}`;

_tr.appendChild(_td);

_td = document.createElement('td');
_td.innerHTML = i.message;

_tr.appendChild(_td); history.appendChild(_tr);

});

}

});

})();

const AddHistory = async() => {

event.preventDefault(); let data

= {};

data.name = document.getElementById('name').value = Appointment.name;
data.phone = document.getElementById('phone').value = Appointment.phone;
data.message = document.getElementById('message').value; data.appoinment =
Appointment._id let options = { method: 'POST', headers:

{ 'Content-Type': 'application/json;charset=utf-8' },

body: JSON.stringify(data) }; await

fetch('/addhistory',

```

```
options).then(e=>e.json()).then(data=>{
  alert(data.data);
})
}
```

Registered Doctors

```
function apotable() { event.preventDefault() let data = null const xhr
=new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) {
console.log(this.responseText); const res =
JSON.parse(this.responseText); var table = '<table id="doctable">
<tr><th>Doctor Id</th><th>Doctor
Name</th><th>Department</th><th>E-mail</th><th>Actions</th></tr>', element
= ""; if (res.data.length > 0) { for (let index = 0; index < res.data.length; index++)
{ element += '<tr><td>' + (index + 1) + '</td><td>' + res.data[index].name +
'</td><td>' + res.data[index].department + '</td><td>' + res.data[index].email +
'</td><td><div class="btn_turf" onclick="deletedoc(\' + res.data[index]._id +
\')"><button>Delete</button></div></td></tr>'; } }
table+=element; table += '</table>'
document.getElementById("doctable").innerHTML = table;
} });

xhr.open("GET", "http://localhost:3000/docview");
xhr.setRequestHeader("Accept", "*/*"); xhr.setRequestHeader("Content-
Type", "application/json"); xhr.send(data); }

document.addEventListener("DOMContentLoaded", () => {
apotable() }); function deletedoc(id) { const xhr = new
XMLHttpRequest(); xhr.withCredentials =
true; xhr.addEventListener("readystatechange",
function () { if (this.readyState === this.DONE) {
console.log(this.responseText); apotable();
alert('Doctor deleted'); } });
```

```

xhr.open("DELETE", "http://localhost:3000/deletedoctor");
xhr.setRequestHeader("Accept", "*/*"); xhr.setRequestHeader("Content-Type",
"application/json"); xhr.send(JSON.stringify({ _id: id }));

}

```

Registered Patients

```

function patable() {

event.preventDefault() let data =
null const xhr = new
XMLHttpRequest();
xhr.withCredentials= true;
xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) {
console.log(this.responseText); const res =
JSON.parse(this.responseText); var table = '<table
id="patable">
<tr><th>Patient Id</th><th>Patient
Name</th><th>E-mail</th><th>Actions</th></tr>', element = ""; if
(res.data.length > 0) { for (let index = 0; index < res.data.length; index++)
{ element += '<tr><td>' + (index + 1) + '</td><td>' + res.data[index].name +
'</td><td>' + res.data[index].email + '</td><td><div class="btn_turf"
onclick="deletpat(' + res.data[index]._id +
'\")"><button>Delete</button></div></td></tr>';
} } table += element; table += '</table>'

document.getElementById("patable").innerHTML = table;

} });

xhr.open("GET", "http://localhost:3000/patview");
xhr.setRequestHeader("Accept", "*/*"); xhr.setRequestHeader("Content-
Type", "application/json"); xhr.send(data); }

document.addEventListener("DOMContentLoaded", () => {

patable() }); function deletpat(id)

```

```

{ const xhr = new
XMLHttpRequest();    xhr.withCredentials =
true;
xhr.addEventListener("readystatechange",
function () { if (this.readyState ===
this.DONE) { console.log(this.responseText);
pattable(); alert('patient Deleted'); } });
xhr.open("DELETE",
"http://localhost:3000/deletepatient");
xhr.setRequestHeader("Accept",    "*/*");
xhr.setRequestHeader("Content-Type",
"application/json"); xhr.send(JSON.stringify({
_id: id })); }

```

Doctor View Appointments

```

Appointments(); function Appointments() { const xhr = new
XMLHttpRequest(); xhr.withCredentials = true; const
Appointments      =      document.getElementById("Appointments");
xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { const res =
JSON.parse(this.responseText); res.data.forEach((i) => { const _tr
= document.createElement('tr'); let _td = document.createElement('td');
_td.innerText = i.name;
_tr.appendChild(_td);
_td = document.createElement('td');
_td.innerText = i.phone;
_tr.appendChild(_td);
_td = document.createElement('td');
_td.innerText = i.email;
_tr.appendChild(_td);
_td = document.createElement('td');

```

```

_td.innerText = i.time;

_tr.appendChild(_td);
_td = document.createElement('td');

_td.innerText = i.date;

_tr.appendChild(_td);

_td = document.createElement('td');

_td.innerText = i.message;

_tr.appendChild(_td);

_td = document.createElement('td');

_td.innerHTML = `<button class="btn btn-light" onclick="window.location.href
= 'patienthis.html?id=${i._id}'">View history</button>`;

_tr.appendChild(_td);

Appointments.appendChild(_tr);

});

}

});

const doctor = JSON.parse(localStorage.user)._id;      xhr.open("GET",
"http://localhost:3000/getappointments?doctor=" +doctor);
xhr.setRequestHeader("Accept", "/"); xhr.setRequestHeader("Content-
Type", "application/json"); xhr.send(null);

}

```

TESTING

Testing is the process of detecting errors. Testing performs a very critical role for quality assurance and for ensuring the reliability of software. The results of testing are used later on during maintenance also. The aim of testing is often to demonstrate that a program works by showing that it has no errors. The basic purpose of testing phase is to detect the errors that may be present in the program. Hence one should not start testing with the intent of showing that a program works, but the intent should be to show that a program doesn't work. Testing is the process of executing a program with the intent of finding errors.

Unit Testing

Unit testing focuses verification effort on the smallest unit of software i.e., the module. Using the detailed design and the process specifications, unit testing is done to uncover errors within the boundary of the module. All modules must be successful in the unit test before the integration testing starts. In this project, each service can be thought of a module. There are 3 modules like Admin, Patient, and Doctor. While developing the modules, unit testing is done to ensure that each module works without any error. The inputs are validated after accepting from the user. In this project, developer tests the programs up as system. Software units in a system are the modules and routines that are assembled and integrated to form a specific function. Unit testing is first done on modules, independent of one another to locate errors. This enables to detect errors. Through these, errors resulting from interaction between modules are initially avoided. Black Box Testing: In the black box testing, test cases are designed from the examination of the input/output values only and no knowledge of design or code is required. The main approaches are:

- Equivalent class partitioning
- Boundary value analysis
- White Box Testing: White box testing is an important type of unit testing. A large number of white box testing strategies exist. White box testing strategies are:
- Fault based testing
- Coverage Based testing

Integration Testing

Integration testing is a type of software testing in which the different units, modules or components of a software application are tested as a combined entity. However, these modules may be coded by different programmers. The aim of integration testing is to test the

interfaces between the modules and expose any defects that may arise when these components are integrated and need to interact with each other.

Types of test cases

Integration test case: Here we execute test cases which just tell about the connectivity from one module to other module and integrating one application to other application and how the application moves from parent node to child no demand vice versa.

Functional Test case: Here we execute test cases which tell about the functionality of the application and talks about the desired output to be seen. Internally we have different type of test cases when we write here like range, output values, BVA, ECP and so on. We give input and expect some output according to the SRS. Here we check individual module completely by checking each and every tab, text box, and buttons and so on. NonFunctional Test case: Test cases related to user friendliness like font, image, color, how easy to navigate etc., performance related, security related comes under here. User Acceptance test case: These test cases are crucial and very important to client-side people, because these test case talks about these business and approach of the application to complete a particular client task, which is also called as End to End Business scenario test case. Here we won't be doing testing related to UI, Functional or Non-Functional, we talk about business and scenario for which the application is made for.

System Testing

Here the entire software system is tested. The reference document for this process is the requirements document, and the goal is to see if software meets its requirements. The entire software has been tested against requirements of project and it is checked whether all requirements of project have been satisfied or not.

Alpha Test: The first test of newly developed, when the first round of bugs has been fixed, the product goes with actual users for testing. For custom software the customer may be invited into the vendor's facilities for an alpha test to ensure the client's vision has been interpreted properly by the developer.

Beta Test: A test of new or revised software application that is performed by users at their facilities under normal operating conditions. Beta testing follows alpha testing. Vendors of packaged software often offer their customers the opportunity of beta testing new releases or versions and the beta testing of elaborate products such as operating systems can take months. Acceptance testing: Testing is performed by the Client of the application to determine whether the application is developed as per the requirements specified by him/her. It is performed within the development of the organization or at the client site. In our project hospital management system, the developer uses unit testing to ensure the overall working of the system.

SECURITY, BACKUP AND RECOVERY

Security:

The described application operates through a simplified yet essential authorization mechanism centered around usernames and passwords. Users are required to register with their unique username and password combination. Access to the application is exclusively granted to registered users, and only through their respective accounts. An integral aspect of this system is the presence of an administrative role. Administrators are tasked with reviewing user registration requests and assessing the authenticity of the provided data, including usernames and potentially additional information. Based on this evaluation, administrators can either approve or decline user requests. This approval process ensures that the application remains secure and is accessed only by legitimate users.

By implementing this straightforward approach, the application effectively establishes a secure user management system, comprising user registration, authentication, and administrative approval. This structure guarantees that access to the application's features is restricted to authorized users, with administrative oversight maintaining data integrity and security.

Backup :

In this application, user support is a pivotal aspect of ensuring a seamless and userfriendly experience. Whenever a user encounters a problem or issue while using the application, a robust support system is in place to assist them in resolving it promptly and effectively. This support may be offered through various channels, such as email, a dedicated helpdesk, or even a chat support feature within the application itself. The goal is to provide users with accessible and responsive assistance to address their concerns, answer their queries, and guide them through troubleshooting steps.

Furthermore, to safeguard the long-term sustainability and security of the application, a backup source code is diligently maintained. This means that a copy of the application's source code is securely stored and regularly updated. This backup serves as an invaluable resource in case of unexpected issues, such as data loss, software bugs, or system failures. Having access to the source code ensures that the application can be restored and improved if necessary, contributing to its reliability and resilience over time. Overall, this comprehensive approach underscores the commitment to user satisfaction and the application's continued functionality and stability.

Recovery:

In the event of an application crash, the user experience remains a top priority, and the system is designed to be resilient and user-friendly. The initial response to such a situation is often as simple as refreshing the application. This quick action can effectively resolve minor glitches, allowing users to get back online and continue using the application without much disruption. However, recognizing that not all issues can be resolved by a simple refresh, the system is equipped with other recovery options. These options are designed to address more severe or persistent problems. Depending on the nature of the issue, users may be guided through troubleshooting steps, offered the option to report the problem to customer support, or provided with an automatic error reporting mechanism that helps the development team diagnose and fix the issue promptly. The presence of multiple recovery options ensures that the application can handle a variety of situations and provide users with a reliable and robust experience. It reflects a commitment to minimizing downtime and disruptions while prioritizing user satisfaction and system stability.

FUTURE ENHANCEMENT

In future, the system can be extended so that it will not only be specific to a doctor and patient but also can be implemented over various hospitals comprising of numerous doctors and patients. Inter hospital data sharing can also be enabled in future so that the process of referring a patient to another hospital will be easy. We will host the platform on online servers to make it accessible worldwide. Now we can only book appointments through this system. We can incorporate video conferencing apps like google meet into our system so that people can consult their respective doctors within the comfort of their home. We can provide SMS alerts in future regarding upcoming appointments. We can incorporate the provision of online bill payments through various services like credit / debit card, UPI, internet banking so that the patients can pay their bills directly without contacting the doctor or other officials. Now the patient has to go to the respective doctor in order to get their prescribed medicines. We can introduce a provision similar to the working of online shopping sites like amazon within our project so that the patient can buy their prescribed medicines online and it will be delivered to them as soon as possible. In future, the patient can also rate a doctor based on their consultation experience and it will be useful for other patients so that they can choose their doctor wisely with utmost confidence. We can provide time slots for booking an appointment so that only one patient can consult a doctor within a particular time slot in future so that it will avoid a rush of patients during a particular time slot. We can also show that if a doctor / patient is online or offline and can also provide a chat box provision in future so that the patient can ask their doubts or queries whenever the respective doctor is online.

SOFTWARE MAINTENANCE

5 types of maintenance have been distinguished, which are differentiated by the nature of the tasks that they include:

Corrective maintenance: The set of tasks is destined to correct the defects to be found in the different equipment and that are communicated to the maintenance department by users of the same equipment.

Preventive Maintenance: Its mission is to maintain a level of certain service on equipment, programming the interventions of their vulnerabilities in the most opportune time. It is used to be a systematic character, that is, the equipment is inspected even if it has not given any symptoms of having a problem.

Predictive Maintenance: It pursues constantly know and report the status and operational capacity of the installations by knowing the values of certain variables, which represent such state and operational ability. To apply this maintenance, it is necessary to identify physical variables (temperature, vibration, power consumption, etc.) and to understand which type of variation is indicative of problems that may be appearing on the equipment. This maintenance it is the most technical, since it requires advanced technical resources, and at times of strong mathematical, physical and / or technical knowledge.

Zero Hours Maintenance (Overhaul): The set of tasks whose goal is to review the equipment at scheduled intervals before appearing any failure, either when the reliability of the equipment has decreased considerably so it is risky to make forecasts of production capacity. This review is based on leaving the equipment to zero hours of operation, that is, as if the equipment were new. These reviews will replace or repair all items subject to wear. The aim is to ensure, with high probability, a good working time fixed in advance.

Periodic maintenance (Time Based Maintenance TBM): The basic maintenance of equipment made by the users of it. It consists of a series of elementary tasks (data collections, visual inspections, cleaning, and lubrication, retightening screws) for which no extensive training is necessary, but perhaps on brief training. This type of maintenance is the based on TPM (Total Productive Maintenance).

CONCLUSION

We propose to develop a software application to aid in patient data management and viewing. The system is aimed to help doctors to enter as well as view patient history as well as other patient details. Our system is a standalone system that can be installed on doctor desktop or phone to be used for further login. On installation, the application allows a doctor to open application and enter the details of any patient that undertakes his service. The application allows doctor to insert various data fields regarding a patient. The system saves this patient related data. The doctor may check the details whenever needed. The application allows doctor to search patients by name as well as date. The proposed application is an easy-to-use application that helps users store their personal data on a server. Our system can be used in public places with Wi-Fi. We are working on this system to make it available offline as well. Our system will be made available in all app stores like google play, apple store etc. We are planning to make people aware about this app by providing meet up, awareness programs etc. As it keeps patient and doctor records on a single platform, it is therefore easily reliable and efficient. This app is designed for a short time has many features that require the user according to their requirement. The app requires less space and is more powerful compared to other existing applications. There is an added range of improvements to this application for user requirements. More features can be added to this app in the future like video conferencing, online medicine shopping, rating doctors regarding their services etc.

BIBLIOGRAPHY

BOOK REFERENCE

- 1) Rajib Mall, “Fundamentals of software engineering” , 2018, PHI Learning.
- 2) Shari Lawrence Pfleeger, “Software engineering Theory and Practice”, 2010, Pearson.
- 3) Brad Green and Shyam Seshadri, “AngularJS”, 2013, O’Reilly.
- 4) Simon Holmes and Clive Harber, “Getting MEAN with Mongo, Express, Angular, and Node”, 2019, Manning Publications

WEBSITE REFERENCE

- 1) <https://nevonprojects.com/android-patient-tracker>
- 2) <https://devpost.com/software/patient-tracker-project>
- 3) <https://github.com/topics/hospital-management>
- 4) <https://tmc.kerala.gov.in/>
- 5) <https://gg-hospital.com/>